

Dictionary-Guided Mutation Operators for Automated HDL Repair

Maisha Mastora

Department of Electrical and Computer Engineering
University of New Hampshire
Durham, NH, USA
maisha.mastora@unh.edu

Dean Sullivan

Department of Electrical and Computer Engineering
University of New Hampshire
Durham, NH, USA
dean.sullivan@unh.edu

Abstract—Automated repair of Hardware Description Language (HDL) designs remains challenging due to the large search space of candidate repairs and the strict syntactic and semantic constraints imposed by HDL grammars. Generic mutation strategies overwhelmingly generate syntactically invalid candidates that waste compilation and simulation budget, while synthesis-driven and template-based approaches impose their own constraints on generality and portability. In this paper, we propose a dictionary-guided HDL repair system that combines ANTLR-derived DUT-specific mutation vocabularies with a simulation-divergence fault localization (FL) module. The mutation operator applies category-constrained token substitutions, insertions, and deletions directly to Verilog source via regex-based matching, without requiring AST manipulation or synthesis. The FL module identifies diverging output wires from a single simulation run and scores source lines by structural proximity to those signals, directing the mutation search toward high-suspicion regions. A deterministic targeted sweep exhausts all dictionary mutations on the highest-scored lines before falling back to a genetic programming (GP) search. Evaluated on the CirFix benchmark suite across six design under test (DUT) families, the proposed approach produces correct oracle-passing repairs on 14 bug variants, including a 6-edit multi-bug instance that CirFix cannot repair, and achieves an $18\times$ speedup over CirFix on a two-edit benchmark variant. These results indicate that dictionary-constrained mutation operators, combined with lightweight simulation-divergence FL, are a practical and competitive approach to automated HDL repair for common bug classes without formal analysis or synthesis dependencies.

Index Terms—HDL repair, automated program repair, Verilog, mutation operators, fault localization.

I. INTRODUCTION

Automated Program Repair (APR) has been studied extensively in the software domain, where techniques such as symbolic reasoning, constraint solving, and heuristic search have demonstrated the ability to produce correct patches with minimal human intervention [1]. A parallel line of work treats repair as a search problem guided by an oracle, where the objective is to navigate a large space of program transformations toward variants that satisfy the required correctness specification. From this perspective, the quality of the search is

determined not only by the oracle, but by the mutation operator that generates candidates for evaluation.

A fundamental limitation of text-level or byte-level mutation is poor alignment with program syntax. For languages with rich formal grammars — such as HDLs — unconstrained token edits produce syntactically invalid candidates at a high rate, exhausting compilation and simulation budget on variants that cannot contribute to repair. Prior work in structured fuzzing has demonstrated that token-aware mutation, which respects the syntactic structure of the input language, substantially improves exploration efficiency [2], [3]. The same principle applies directly to HDL repair: constraining mutations to grammar-permitted, design-local tokens increases the density of valid and behaviorally plausible candidates in the search space.

HDLs are particularly well-suited to this approach. Their syntax and semantics are defined by formal grammars with explicit token classes, operator categories, and contextual constraints. These structures can be extracted statically from any design to construct a DUT-specific mutation vocabulary without simulation or synthesis. The vocabulary defines precisely what substitutions, insertions, and deletions are structurally meaningful for each token category in the design under repair.

In this paper, we present a complete HDL repair system built around this principle. The core contributions are: (i) an ANTLR-based dictionary construction pipeline that extracts DUT-specific structural and numeric token vocabularies from the Verilog grammar and the buggy source; (ii) a simulation-divergence FL module that scores source lines by proximity to diverging output wires without AST parsing; (iii) a category-constrained mutation operator that directs substitutions, insertions, and deletions toward high-suspicion regions; and (iv) a targeted deterministic sweep that exhausts single-edit repairs before engaging a GP fallback, enabling multi-edit repair through iterative partial-fix accumulation.

We evaluate our work on the CirFix benchmark suite [4] across six DUT families, comparing against CirFix as a baseline. Our approach repairs 14 bug variants, including variants that CirFix cannot fix, and achieves substantially lower evaluation counts on several benchmarks. The results suggest that dictionary-constrained mutation, without formal analysis or synthesis, is a viable and competitive strategy for HDL repair

across a range of common bug classes. Crucially, the mutation vocabulary is derived automatically from each design, requiring no manual repair templates.

II. BACKGROUND AND RELATED WORK

A. HDL Repair and Oracle-Based Evaluation

Automated repair of hardware design code differs fundamentally from software repair due to the concurrent, stateful, and timing-dependent nature of HDLs. Correctness is typically defined through simulation behavior against reference input-output traces, rather than sequential control flow properties. Consequently, HDL repair systems rely on explicit correctness oracles — testbenches or golden traces — to evaluate candidate fixes [4], [5].

CirFix [4] is the most directly comparable existing work. It applies a genetic programming search over AST-level mutations, guided by a weighted fitness function that scores candidate output traces against the oracle. CirFix operates on the full design AST, applying insertion, deletion, and replacement at the statement and expression level. While effective on several benchmark classes, CirFix incurs high per-evaluation cost due to AST manipulation, and its mutation operators are not constrained by the DUT’s own token vocabulary. Certain fixes further require design-specific template declarations, limiting generalization across diverse designs.

RTL-Repair [5] takes a symbolic approach, encoding the repair problem as a bounded synthesis query over a localized fault window. This yields fast and correct repairs within its synthesis scope, but requires synthesizability of the target design and is limited to patches within the bounded template. SRepair [6] applies symbolic regression over signal traces to infer repair expressions, and STRIDER [7] uses signal transition patterns to guide repair for specific defect classes. Both improve repair coverage for their target bug types but introduce synthesis or formal analysis dependencies that the proposed approach intentionally avoids.

B. Fault Localization for Hardware

Fault localization (FL) for HDL has been approached through both learning-based and simulation-based methods. Hu and Liu [8] use deep learning over execution coverage and contextual token representations to rank suspicious statements, achieving strong localization accuracy but leaving patch synthesis to a separate stage. STRIDER [7] and related approaches integrate FL with repair by using signal-level divergence to constrain the search space.

The FL module proposed in this work is closest in spirit to simulation-divergence approaches: it identifies the first timestep at which output wires diverge from the oracle, then traces backward through the source using regex patterns to score lines by structural relationship to the diverging signal. This requires no AST, no learning, and only a single simulation run of the buggy design.

C. LLM-Based Approaches

Large language model-based approaches have recently been explored for hardware debugging [9]. These methods leverage pre-trained models to suggest patches from natural language or code context. While promising in coverage, they generally lack syntactic validity guarantees and are difficult to deploy in deterministic, oracle-constrained repair pipelines. The proposed system operates entirely within a verifiable, simulation-grounded framework.

D. Structured Mutation in Software APR

Token-level fuzzing [2], [3] has demonstrated that mutation operators respecting input language structure substantially outperform byte-level approaches in generating valid, coverage-increasing inputs. Kim et al. [1] formalize the connection between fuzzing-style search and APR, motivating oracle-guided mutation as a repair strategy. The proposed approach extends this line of reasoning to HDL repair, where the grammar structure is particularly rich and the cost of invalid candidates — compilation and simulation — is high.

III. SYSTEM DESIGN

A. Dictionary Construction

As illustrated in Fig. 1, the repair pipeline begins by constructing a DUT-specific mutation vocabulary from the buggy Verilog source. Dictionary construction is performed once per DUT before any repair attempt, using ANTLR [10] to parse the design and analyze the grammar’s Augmented Transition Network (ATN).

For each token in the buggy source, the ANTLR parse tree records its grammar rule context. We then traverse the ATN from each matched rule’s start state to enumerate all literal tokens reachable within that syntactic context. This produces a *context-aware* token vocabulary: rather than augmenting the design’s tokens with the full set of Verilog keywords, we augment only with tokens that are grammatically valid replacements given the structural role of each source token. The result is two dictionaries per DUT: a *structural dictionary* (STRUCT) containing identifiers, keywords, and operators, and a *numeric dictionary* (NUM) containing valid Verilog constant literals.

Numeric literals are handled separately to preserve semantic validity. Binary literals in the source are used as seeds; for each seed, the NUM dict is populated with Hamming-distance-1 and distance-2 neighbors (single and double bit flips), all-zeros and all-ones variants, and one-position left and right rotations. Plain decimal integers (e.g., loop bounds) are augmented with a small neighborhood of adjacent values. No cross-width or cross-base substitutions are generated, preserving the original literal’s type context.

When multiple buggy variants of the same DUT exist, per-variant dictionaries are merged to produce both a *union* dictionary (covering all tokens seen across variants) and an *intersection* dictionary (covering only tokens common to all variants). The union dictionary is used by default, maximizing vocabulary coverage.

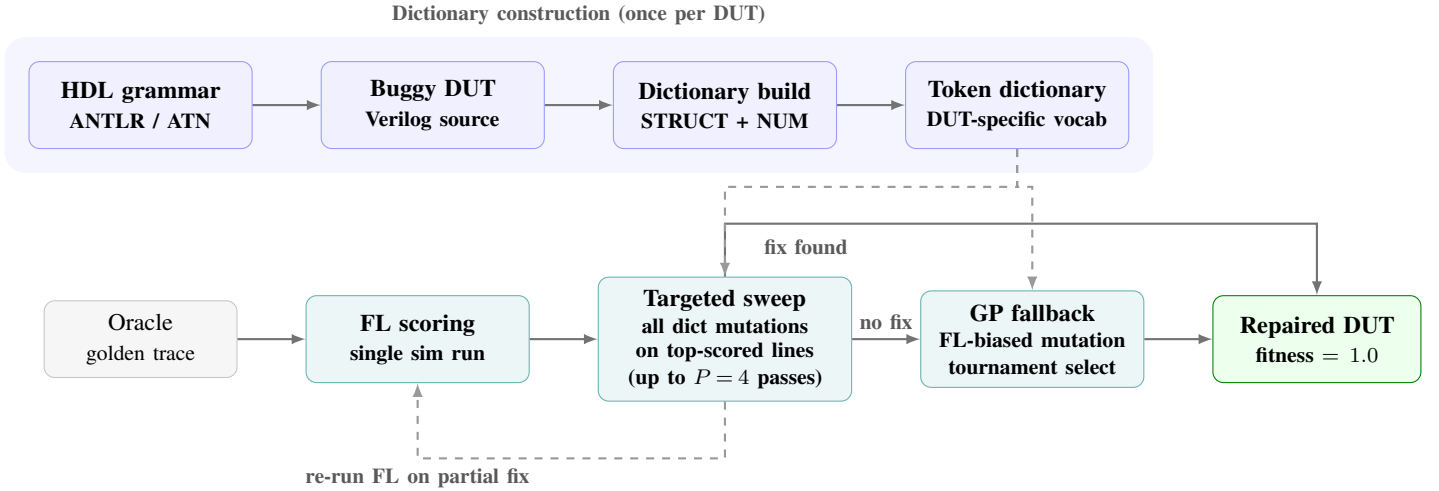


Fig. 1. System overview. The dictionary construction stage (top, shaded) is executed once per DUT and produces a token vocabulary that guides all mutations. The repair pipeline (bottom) runs FL scoring from a single simulation, then exhaustively sweeps top-scored lines; if no fix is found the best partial candidate is passed to GP-based search.

B. Simulation-Divergence Fault Localization

We use simulation-divergence fault localization to identify suspicious lines. The buggy design is simulated once and its output CSV compared against the oracle to identify the first diverging timestep and the set of affected output wires. These wires anchor a backward trace through the source.

Analysis uses only regular expressions on raw Verilog text — no AST or parser. Each line receives a suspiciousness score in $[0.1, 1.0]$ based on its structural relationship to the diverging signals. Scoring proceeds in four passes. Pass A scores any `assign` statement whose LHS includes a diverging wire at 1.0, implicating the full statement up to its terminating semicolon. Pass B locates `always` blocks via `begin/end` depth counting. Blocks driving diverging wires score their header at 0.8 (or 1.0 if the sensitivity list lacks an edge keyword, indicating a missing `posedge/negedge` bug). Pass C handles logic inside these blocks: `if/case` conditions score 0.7–0.9 and procedural assignments score 0.9 (reduced to 0.5 inside detected reset handlers, which rarely contain the functional bug). Pass D first scores `always` blocks driving signals on the RHS of already-implicated lines (≥ 0.7) at 0.8 for headers and conditions, then assigns 0.3 to any line assigning to a secondary signal from high-scoring RHS expressions. All other lines default to 0.1.

The method is hardware-aware: it distinguishes combinational from sequential logic and treats sensitivity-list bugs as a first-class case by inspecting `always@` headers for edge qualifiers. If no divergence is detected, every line receives a uniform score of 0.5 and FL guidance is disabled.

C. Mutation Operators

The mutation operator works directly on Verilog source text using pre-compiled regex patterns. Three primitives are defined:

Substitute: A token present in the source is replaced with a same-category token from the dictionary. Identifiers are further

restricted to replacements that also appear in the source (preserving in-vocabulary semantics). Numeric tokens are drawn from the same-width pool in the NUM dict, with binary tokens sorted by Hamming distance so that single-bit-flip substitutions are sampled with higher probability. Keyword categories (e.g., `posedge/negedge`) are not source-filtered, since the absence of such a token *is* the bug in sensitivity list defects. Substitution selects among numeric and structural candidates with equal probability (50/50 split) to prevent either category from dominating.

Insert: A keyword token is inserted immediately before or after a user identifier occurrence. The insertable pool is restricted to edge keywords (`posedge/negedge`), conditional trail tokens (`else`), and unary logical operators (`!`, `~`). When the insertion anchor is detected inside an `always@(...)` sensitivity list, the pool is further restricted to edge keywords only.

Delete: A token occurrence is removed. Only tokens in non-structural categories are eligible; identifiers, arithmetic/logical operators, block delimiters, module keywords, and net-type declarations are never deleted, as removal would leave syntactically broken or semantically undefined source. When FL line scores are available, all three operators prefer high-suspicion lines: each token in the source is assigned a score equal to the maximum line score across all its occurrences, and tokens are sorted descending by this score before the mutation loop begins. Among multiple occurrences of the selected token, the specific occurrence to mutate is sampled proportionally to the line suspiciousness scores, biasing mutations toward FL-implicated regions without hard-restricting the search.

D. Repair Pipeline

The repair procedure, summarized in Algorithm 1, proceeds in two stages.

Algorithm 1 Repair Procedure

Require: Buggy source s_0 , mutation dictionary $\mathcal{D}$, oracle $\mathcal{O}$

```
1: scores  $\leftarrow$  FL( $s_0, \mathcal{O}$ ) {single buggy simulation}
2: for  $p = 1$  to  $P$  do
3:    $s^* \leftarrow$  TargetedSweep( $s_{\text{cur}}, \mathcal{D}, \text{scores}$ )
4:   if fitness( $s^*$ ) = 1.0 then
5:     return  $s^*$ 
6:   end if
7:   if fitness( $s^*$ ) > fitness( $s_{\text{cur}}$ ) then
8:      $s_{\text{cur}} \leftarrow s^*$ ; scores  $\leftarrow$  FL( $s_{\text{cur}}, \mathcal{O}$ )
9:   else
10:    break
11:  end if
12: end for
13: pop  $\leftarrow$  SeedPopulation( $s_0, \mathcal{D}, \text{scores}$ )
14: for  $g = 1$  to  $G$  do
15:   pop  $\leftarrow$  EvaluateAndSelect(pop,  $\mathcal{D}, \text{scores}$ )
16:   if any individual has fitness 1.0 then
17:     return it
18:   end if
19: end for
20: return best individual seen
```

Stage 1: Targeted deterministic sweep. Prior to stochastic search, the repair procedure exhaustively applies every dictionary mutation to the highest-scored lines. In each pass, every token on the top-scoring line is substituted with every same-category dictionary entry; every edge keyword is inserted before every identifier on that line; and every deletable conditional token is removed from all lines scoring ≥ 0.7 . The sweep also covers all lines scoring ≥ 0.7 with full substitution, applies unary operator insertion ($!$, $\sim$) across top and mid-score lines, and performs identifier and numeric substitution on lines scoring in $[0.3, 0.6)$. Each candidate is evaluated immediately and the sweep exits upon the first fitness-1.0 result.

When no candidate achieves fitness 1.0 but one strictly improves over the current best, FL is re-executed on that improved partial candidate. The updated scores reflect the residual divergence after the partial fix, potentially implicating different lines than the original scoring. The sweep is then re-executed on the updated candidate with the new scores, excluding previously swept lines to avoid redundant work. This repeats for up to P passes (default $P = 4$). Each pass operates on the best candidate produced by the preceding pass, allowing independently discovered single-edit fixes to accumulate. This yields effective multi-edit repair without requiring coordinated multi-token mutations within a single individual — a known difficulty in standard GP-based APR [4]. If all passes are exhausted without a perfect repair, the best partial candidate is retained and control passes to Stage 2.

Stage 2: GP fallback. A population of `POPSIZE` candidates (default 200) is seeded by applying FL-biased single mutations to the original buggy source. Seeding from the original source maintains vocabulary diversity and prevents premature convergence on partial fixes. Each generation applies tournament selection (tournament size $k = 3$) to select parents, then applies a single FL-biased mutation at rate `MUTATION_RATE` (default 0.7). The top 10% of each generation are carried

forward unchanged (elitism). Any candidate achieving fitness 1.0 terminates the search immediately. The search runs for `GENS` generations (default 10) across `RESTARTS` independent restarts (default 5).

Fitness function. Fitness is computed by the CirFix oracle fitness function [4]: each candidate is compiled with `iverilog`, simulated, and its output CSV compared bitwise against the oracle trace. Matching bits contribute +1, mismatches -1, and `x/z` matches contribute ± 2 (weighted). The normalized fitness $f \in [0, 1]$ is returned; candidates that fail compilation receive fitness 0 without simulation.

IV. EXPERIMENTAL EVALUATION

A. Setup

We evaluate the proposed system on the CirFix benchmark suite [4], which provides buggy Verilog designs with corresponding testbenches and golden oracle traces. We use the following six DUT families: `first_counter_overflow`, `fsm_full`, `lshift_reg`, `mux_4_1`, `decoder_3_to_8`, and `flip_flop`. Designs where all variants pass the provided oracle with fitness 1.0 on the original buggy source are excluded, as this indicates insufficient testbench coverage to expose the injected bugs — a known oracle weakness in the benchmark, not a failure of the repair system.

The repair system is configured with `popsiz=200`, `gens=10`, `restarts=5`, `mutation_rate=0.7`, and `dict_type=union`. To ensure a fair comparison under identical conditions, we re-ran the original CirFix implementation on the same machine and simulator used for all experiments. The original CirFix paper used Synopsys VCS, a commercial simulator significantly faster per simulation cycle than `iverilog`. Because we do not have access to VCS, all local runs use `iverilog` for both systems, making wall-clock times directly comparable on our setup. Paper-reported CirFix times are cited where local replication was not feasible, but these are not directly comparable due to the simulator difference and are marked accordingly.

We use *fitness evaluation count* as the primary efficiency metric, since it is machine- and simulator-independent: one evaluation corresponds to a complete compile-simulate-score cycle. Wall-clock times on the same machine and simulator are reported as a secondary metric where available.

All experiments were conducted on a Lenovo ThinkPad P16 Gen 1 (x86-64, Ubuntu 25.10) with Python 3.9 and `iverilog` as the simulator. For non-template bugs requiring large-scale GP search, vanilla CirFix with `popsiz=5000` did not complete within a practical time limit under `iverilog`. This reflects a fundamental difference in search strategy: CirFix requires a large population to compensate for undirected mutation, whereas the proposed system repairs the same bugs with `popsiz=200` because dictionary-guided mutation restricts candidates to semantically plausible token substitutions from the design’s own vocabulary.

B. Results

Table I summarizes repair outcomes across all evaluated variants. Out-of-scope variants — those requiring repairs outside

the current mutation vocabulary (e.g., commented-out code, hex-to-binary base conversions, port declaration changes) — are listed separately and excluded from the repair count.

a) Overall repair count: The proposed system produces oracle-passing repairs on **14 bug variants** across five DUT families, from a total of 23 in-scope variants (61% repair rate). Three additional variants fail due to limitations discussed in Section V, and six are out of scope for the current mutation vocabulary.

b) Multi-edit repair: The `decoder_3_to_8` super_buggy variant contains six simultaneous bugs. The proposed system repairs it in four targeted sweep passes (5374 total evaluations), with each pass accumulating one or more single-edit fixes on the best partial candidate from the preceding pass. CirFix does not repair this variant. The `first_counter` buggy_counter variant (3 edits) is similarly repaired via the iterative targeted sweep in 98 evaluations.

c) Efficiency and unique fixes: On the `decoder_3_to_8` wadden_buggy1 variant, the proposed system requires 776s locally versus the paper-reported 13,984s for CirFix (obtained on VCS) — an **18× speedup**. This comparison is noted with the caveat that the CirFix figure uses a faster commercial simulator; the evaluation count comparison (1951 vs. the thousands required by CirFix’s GP search) is simulator-independent and reflects the same advantage. For non-template bugs, local vanilla CirFix with `popsize=5000` did not finish under `iverilog` within a practical time limit, while the proposed system resolves these with `popsize=200`. Five additional variants marked ^u are repaired by the proposed system but not by CirFix, covering identifier substitution, plain integer, and unary operator bug classes outside CirFix’s mutation template coverage. Our system trades some speed on template-covered bugs for generality: its mutation vocabulary is derived automatically from any design, requiring no manually curated templates.

V. DISCUSSION AND LIMITATIONS

a) Simulator and timing comparability: The original CirFix paper used Synopsys VCS; our evaluation uses `iverilog`, which has higher per-invocation overhead. All local comparisons between the two systems use `iverilog` on the same machine and are internally consistent. Paper-reported CirFix times are cited only where local re-runs were not feasible and are marked * accordingly; these are not directly comparable in wall-clock terms.

b) Oracle quality and spurious repairs: Three `fsm_full` variants are marked as oracle-passing with a weak oracle caveat. The `fsm_full` testbench covers only 39 input combinations, which is insufficient to distinguish semantically correct repairs from spurious oracle-passing patches. The proposed system found oracle-passing repairs for `buggy_num` and `buggy_var` that differ from the intended fix (e.g., `req_0` → `gnt_0` instead of the correct `GNT1` → `IDLE`). The semantically correct fixes also pass the oracle, confirming that the oracle weakness — not the repair search — is the source of ambiguity. This is the standard *overfitting* problem in oracle-guided

APR, well-documented for both software [1] and hardware repair systems [4]. The proposed approach produces oracle-passing repairs; semantic correctness beyond the provided test suite is outside the scope of the repair guarantee.

c) Simultaneous multi-edit repair: The `flip_flop` tff_wadden_buggy2 variant requires a simultaneous swap of two assignments ($q \leq q \leftrightarrow q \leq \sim q$). The FL module correctly scores both buggy lines at 0.9, but the greedy single-edit targeted sweep fails: repairing either line in isolation reduces fitness before the other is corrected. CirFix handles this via AST-level GP, which tolerates fitness valleys. Similarly, the `mux_4_1` wadden_buggy1 variant requires correcting three case labels simultaneously; any single-label fix temporarily reduces fitness below the buggy baseline (0.2667 vs. 0.7333). Both cases represent a fundamental limitation of greedy sequential repair and motivate coordinated multi-token patch generation as a direction for future work.

d) Blocking-to-non-blocking conversion: The `lshift_reg` wadden_buggy1 variant requires converting four blocking assignments (=) to non-blocking (<=). The proposed system achieves a best fitness of 0.8276 — one assignment is correctly converted by the targeted sweep, but the GP search fails to accumulate all four fixes simultaneously, as the repair requires modifying four independent lines at once.

e) Non-minimal patches: In the `lshift_reg` kgoliya_buggy1 variant, the correct single-edit fix (`negedge` → `posedge`) is found in the first targeted sweep pass. However, subsequent passes introduce a spurious additional substitution (`i<8` → `i<10`) that also passes the oracle. The resulting patch is functionally correct but not minimal. The current system does not include a post-processing minimization step; delta debugging [1] over the produced patch would address this.

f) Out-of-scope mutations: Several benchmark variants require repair operations outside the current vocabulary: commented-out code (requiring line reinsertion), port declaration width changes, hex-to-binary base conversions for literal constants not in the NUM dict, and compile-time syntax errors that prevent oracle evaluation. Extending the dictionary construction to cover these classes represents a clear path to broader coverage.

VI. CONCLUSION AND FUTURE WORK

We presented a dictionary-guided HDL repair system combining ANTLR-derived DUT-specific mutation vocabularies with simulation-divergence fault localization. The system operates directly on Verilog source text without AST manipulation, synthesis, or formal analysis, making it portable across designs and independent of synthesizability constraints. The dictionary pipeline, executed once per DUT, bounds the mutation space to grammar-valid, design-specific tokens. The FL module, requiring only a single simulation of the buggy design, scores source lines by structural proximity to diverging output wires and directs both the targeted sweep and the GP fallback toward high-suspicion regions.

Evaluated on the CirFix benchmark suite across six DUT families, the proposed system produces oracle-passing repairs

TABLE I

REPAIR RESULTS ON THE CIRFIX BENCHMARK SUITE. OOS = OUT OF SCOPE. † = ORACLE-PASSING; WEAK ORACLE (SECTION V). ^U = UNIQUE FIX NOT REPAIRED BY CIRFIX. TIMES IN SECONDS; CIRFIX TIMES ARE LOCAL IVERILOG RE-RUNS WHERE AVAILABLE, OTHERWISE PAPER-REPORTED VCS TIMES* (NOT DIRECTLY COMPARABLE). — = NOT ATTEMPTED OR DID NOT FINISH.

DUT	Variant	Bug Type	Result	CirFix
<i>first_counter</i>	buggy_overflow	Numeric sub: 1'b0→1'b1	✓ 0.56s	✓ 0.4s*
	wadden_buggy1	Missing posedge in sensitivity list	✓ 2.04s	✓ 2.64s
	wadden_buggy2	Spurious else branch	✓ 7.93s	✓ 2.84s
	buggy_counter	3-edit: +2→+1, >=>==, bit fix	✓ 20.9s ^U	×
	kgoliya_buggy1	Commented-out reset line	OOS	—
	buggy_all	3 simultaneous edits (combined)	OOS	—
<i>lshift_reg</i>	kgoliya_buggy1	Edge keyword: negedge→posedge	✓ 38.7s	✓ 7.8s
	buggy_num	Plain int: i=2→i=0	✓ 13.3s ^U	—
	buggy_var	Identifier sub: rstn→load_en	✓ 25.1s	✓ 33.7s
	wadden_buggy2	Missing ! before rstn	✓ 181s ^U	—
	wadden_buggy1	4 blocking =>→<=	× (best 0.83)	✓ 14.6s
<i>mux_4_1</i>	buggy_var	Identifier sub: a→b	✓ 14.5s ^U	×
	wadden_buggy1	3 case labels wrong (simultaneous)	× (best 0.73)	×
	kgoliya_buggy1	Missing [3:0] on output port	OOS	—
	wadden_buggy2	Hex→binary conversion (4 edits)	OOS	—
<i>decoder_3_to_8</i>	buggy_var	Identifier sub: en→C	✓ 34.1s ^U	×
	wadden_buggy1	2-edit numeric	✓ 776s	✓ 13984s*
	buggy_num	Numeric sub: 8'b1111_→1111→...1110	✓ 484s ^U	—
	super_buggy	6-edit multi-bug (4-pass sweep)	✓ 2110s ^U	×
	wadden_buggy2	Missing underscores in literals	OOS	—
	kgoliya_buggy1	Syntax error: assign <=	OOS	—
<i>fsm_full</i>	buggy_num	Numeric sub: 1'b1→1'b0	✓ †	✓
	buggy_var	Identifier sub: GNT1→IDLE	✓ †	✓
	ssscrazy_buggy1	Missing state in sensitivity list	✓ †	✓
	ssscrazy_buggy2	Missing assignment + default case	× (best 0.99)	×
	wadden_buggy1	Commented-out case branch	OOS	—
<i>flip_flop</i>	tff_wadden_buggy1	Missing ! before rstn	✓ 8.78s	✓ 7.8s
	tff_wadden_buggy2	Swapped branches (simultaneous 2-edit)	× (best 0.78)	✓ 924s*

on 14 bug variants — including 3-edit and 6-edit multi-bug instances that CirFix cannot repair — while achieving up to an 18× speedup on shared benchmarks. These results indicate that dictionary-constrained mutation directed by lightweight simulation-divergence FL is a practical and competitive approach to HDL repair for common bug classes, without requiring formal analysis or synthesis.

Future work includes coordinated multi-token patch operators for simultaneous multi-edit bugs, post-repair minimization via delta debugging to eliminate spurious edits, extended numeric dictionary coverage for base-agnostic and signed arithmetic substitutions, and weaker oracle dependence through partial or property-based specifications.

REFERENCES

- [1] Y. Kim, S. Han, A. Y. Khamit, and J. Yi, “Automated program repair from fuzzing perspective,” in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 854–866.
- [2] B. Mathis, R. Gopinath, and A. Zeller, “Learning input tokens for effective fuzzing,” in *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, 2020, pp. 27–37.
- [3] C. Salls, C. Jindal, J. Corina, C. Kruegel, and G. Vigna, “{Token-Level} fuzzing,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2795–2809.
- [4] H. Ahmad, Y. Huang, and W. Weimer, “Cirfix: Automatically repairing defects in hardware design code,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 990–1003.
- [5] K. Laeuffer, B. Fajardo, A. Ahuja, V. Iyer, B. Nikolic, and K. Sen, “Rtl-repair: Fast symbolic repair of hardware design code,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 867–881.
- [6] Z. Liu, D. Yang, X. Mao, J. He, G. Zhang, Y. Lei, and J. Wu, “Srepair: Symbolic regression-based repair for hardware design code,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [7] D. Yang, J. He, X. Mao, T. Li, Y. Lei, X. Yi, and J. Wu, “Strider: Signal value transition-guided defect repair for hdl programming assignments,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1594–1607, 2023.
- [8] J. Hu and Z. Liu, “Context aware deep learning-based fault localization for hardware design code,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [9] W. Fu, S. Li, Y. Zhao, K. Yang, X. Zhang, Y. Jin, and X. Guo, “A generalize hardware debugging approach for large language models semi-synthetic, datasets,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2024.

[10] ANTLR Project, “Antlr4,” <https://github.com/antlr/antlr4>, 2024, accessed: 2026-01.